

RAPPORT DE GESTION DE PROJET — Puls-Events

Table des matières

Introduction	3
Contexte et objectif du projet	3
1. Analyse et synthèse des besoins	3
1.1 Objectifs métiers	3
1.2 Contraintes techniques	4
1.3 Analyse des utilisateurs cibles	4
1.4 Cas d’usage principaux	4
1.5 Parties prenantes	4
1.6 Risques et hypothèses	5
2. Plan de projet	5
2.1 Jalons	5
2.2 Échéanciers	5
2.3 Livrables	6
2.4 Gouvernance	6
3. Macro backlog	6
EPIC 1 — Ingestion et preprocessing	6
EPIC 2 — Vectorisation et FAISS	6
EPIC 3 — RAG et LangChain	6
EPIC 4 — Chatbot et Frontend	6
EPIC 5 — Mémoire conversationnelle	7
EPIC 6 — Géolocalisation	7
EPIC 7 — Recherche web temps réel	7
EPIC 8 — Monitoring	7
4. Architecture technique détaillée	7
4.1 Architecture cloud (AWS)	7
Composants	7
4.1.1 Scalabilité de l’index vectoriel	8
4.2 Justification du cloud provider	9
4.3 Stratégie de déploiement	9
4.4 Monitoring	9
4.5 Sécurité et RGPD	9
5. Estimation des coûts	9

5.1 BUILD	9
5.2 OPEX.....	10
5.3 Optimisation budgétaire	10
5.4 KPIs	10
6. Bilan.....	10
6.1 Du POC au MVP	10
6.2 Justification des choix	10
6.3 Défis rencontrés	10
Conclusion.....	11
Annexes	11
Portfolio GitHub.....	11
Liens utiles	11

Introduction

Puls Events est une plateforme web permettant aux utilisateurs de découvrir des événements culturels en temps réel, personnalisés selon leurs préférences (lieu, période, thématique). Un premier POC a démontré la faisabilité d'un moteur de recherche sémantique basé sur un pipeline RAG (Retrieval Augmented Generation) utilisant FAISS, Mistral AI et LangChain.

L'objectif de ce projet est de transformer ce POC en un MVP scalable, maintenable, sécurisé et cloud-ready, intégrable dans la plateforme Puls Events, et capable de fournir des recommandations personnalisées, contextualisées (géolocalisation) et robustes. Le MVP inclut également un frontend léger basé sur Streamlit pour permettre une démonstration rapide et une validation utilisateur.

Ce rapport présente :

- l'analyse des besoins métiers et techniques,
- l'architecture cloud détaillée,
- le backlog et la roadmap,
- le plan de projet,
- l'estimation des coûts (BUILD + OPEX),
- les recommandations stratégiques pour la mise en production.

Contexte et objectif du projet

Le marché de l'événementiel évolue rapidement, avec une demande croissante pour des expériences personnalisées et des outils intelligents basés sur l'IA. Puls Events souhaite intégrer un chatbot intelligent capable de :

- comprendre le langage naturel,
- rechercher des événements pertinents,
- personnaliser les réponses selon l'utilisateur,
- s'adapter à la localisation,
- intégrer des données en temps réel.

Le MVP doit être :

- scalable,
- maintenable,
- sécurisé,
- économique,
- prêt pour la production,
- doté d'un frontend simple (Streamlit) pour les tests utilisateurs.

1. Analyse et synthèse des besoins

1.1 Objectifs métiers

Les objectifs métiers du projet Puls Events s'inscrivent dans une volonté globale d'améliorer l'expérience utilisateur et de renforcer la valeur ajoutée de la plateforme. L'entreprise souhaite proposer un service capable de comprendre les intentions des utilisateurs, de réduire le temps nécessaire pour trouver un événement pertinent et d'augmenter la satisfaction globale. Le chatbot doit permettre une interaction fluide, naturelle et personnalisée, en tenant compte des préférences individuelles, de l'historique de navigation et du contexte géographique. Enfin, le projet vise à poser les bases d'une architecture scalable et industrialisable, afin de préparer une future montée en charge et une éventuelle commercialisation du produit.

1.2 Contraintes techniques

Le projet doit composer avec plusieurs contraintes techniques importantes. Les données issues d'OpenAgenda sont hétérogènes, parfois incomplètes ou mal structurées, ce qui nécessite un travail de normalisation et de nettoyage. Le volume de données peut varier fortement selon les périodes (vacances, festivals, week-ends), ce qui impose une architecture capable d'absorber des pics de charge. L'utilisation de Mistral API pour la génération de texte implique une dépendance externe, avec une latence variable et un coût d'inférence proportionnel au nombre de requêtes. Enfin, le moteur RAG nécessite un stockage vectoriel performant et scalable, capable de supporter des recherches rapides même en cas d'augmentation du volume de données ou du trafic utilisateur.

1.3 Analyse des utilisateurs cibles

Les utilisateurs de Puls Events présentent des profils variés, allant des habitants locaux aux touristes, en passant par les familles, les étudiants ou les professionnels. Chaque catégorie d'utilisateur possède des besoins spécifiques : rapidité de recherche, filtrage par budget, découverte culturelle, activités adaptées aux enfants ou encore événements professionnels. Le chatbot doit être capable de comprendre ces besoins et d'y répondre de manière contextualisée, en s'appuyant sur les données disponibles et sur les préférences exprimées par l'utilisateur.

Utilisateur	Besoin	Exemple
Habitants de Paris	Activités rapides à trouver	« Que faire ce week-end ? »
Familles	Activités enfants	« Événements famille 2025 »
Étudiants	Gratuit / petit budget	« Gratuit Paris 2025 »
Touristes	Découvrir la ville	« Que faire dans le 13e ? »
Professionnels	Networking	« Événements tech 2025 »

1.4 Cas d'usage principaux

Les cas d'usage identifiés couvrent l'ensemble des interactions attendues entre l'utilisateur et le chatbot. Il doit être possible de rechercher un événement selon une date précise, un arrondissement, une thématique ou un critère tarifaire. Le système doit également être capable de proposer des recommandations personnalisées, basées sur l'historique ou les préférences déclarées. Enfin, l'expérience doit être conversationnelle, permettant à l'utilisateur de poser des questions successives tout en conservant le contexte de la discussion.

1.5 Parties prenantes

Le projet implique plusieurs parties prenantes internes. La direction et l'équipe produit définissent les objectifs stratégiques et les priorités fonctionnelles. L'équipe Data et Tech est responsable de la conception, de l'implémentation et de la mise en production du MVP. L'équipe Marketing exploite les données d'usage pour améliorer l'engagement et orienter les évolutions futures. Les utilisateurs finaux constituent la cible principale et leurs retours guideront les itérations du produit.

1.6 Risques et hypothèses

Plusieurs risques ont été identifiés. La qualité des données OpenAgenda peut impacter la pertinence des recommandations. La latence de l'API Mistral peut dégrader l'expérience utilisateur en cas de forte sollicitation. Les coûts liés aux appels LLM peuvent augmenter rapidement en fonction du trafic. Enfin, la scalabilité de l'index vectoriel FAISS doit être maîtrisée pour éviter les ralentissements. Les hypothèses retenues sont que le volume initial de données reste compatible avec FAISS, que l'API OpenAgenda demeure stable et que le MVP sera d'abord exposé à un trafic modéré.

2. Plan de projet

2.1 Jalons

Le projet est structuré en sept jalons successifs permettant de valider progressivement chaque composant du MVP. La première étape consiste à valider le design global de l'architecture et des choix techniques. Les semaines suivantes sont consacrées à l'ingestion des données, à la génération des embeddings, à la construction de l'index FAISS et à l'implémentation du pipeline RAG. Une fois le backend fonctionnel, les fonctionnalités de mémoire conversationnelle et de géolocalisation sont ajoutées. Le monitoring est ensuite mis en place afin d'assurer la stabilité et la performance du système. Enfin, un frontend Streamlit est développé pour permettre une démonstration du MVP avant son déploiement final.

Jalons	Description	Date
M0	Validation du design	S1
M1	Ingestion et preprocessing	S2
M2	Embeddings et FAISS	S3
M3	RAG et chatbot backend	S4
M4	Mémoire et géolocalisation	S5
M5	Monitoring	S6
M6	Frontend Streamlit	S7
M7	Déploiement MVP	S8

2.2 Échéanciers

Le projet s'étend sur une durée totale de huit semaines, organisée selon une méthodologie Agile de type Scrum. Chaque sprint dure une semaine et se conclut par une démonstration interne permettant de valider les avancées et d'ajuster les priorités si nécessaire. Cette organisation garantit une progression régulière, une visibilité continue pour les parties prenantes et une capacité d'adaptation en fonction des retours.

2.3 Livrables

Les livrables attendus couvrent l'ensemble des composants nécessaires à la mise en production du MVP. Ils incluent l'architecture cloud détaillée, le pipeline d'ingestion, la base vectorielle FAISS, le pipeline RAG, le backend du chatbot, le frontend Streamlit, ainsi que les outils de monitoring. Des livrables documentaires sont également requis : documentation technique, rapport de gestion de projet, portfolio GitHub et support de présentation.

2.4 Gouvernance

La gouvernance du projet repose sur un suivi rigoureux via GitHub Projects, permettant de visualiser l'avancement des tâches et d'assurer la coordination entre les intervenants. Une réunion quotidienne courte (daily) permet de synchroniser l'équipe et de lever rapidement les obstacles. Chaque fin de semaine, une revue de sprint est organisée pour présenter les résultats et recueillir les retours. Une rétrospective bi-hebdomadaire permet d'améliorer continuellement les processus et la collaboration.

3. Macro backlog

EPIC 1 — Ingestion et preprocessing

User Story	Priorité	Complexité	Risques	Mitigation
Ingestion OpenAgenda	Must	Medium	API lente	Cache local
Nettoyage et normalisation	Must	Medium	HTML sale	BeautifulSoup
Chunking 300–500 tokens	Must	Low	Perte contexte	Overlap

EPIC 2 — Vectorisation et FAISS

User Story	Priorité	Complexité
Embeddings Mistral	Must	Medium
Construction index FAISS	Must	Medium
Sauvegarde index S3	Must	Low
Chargement FAISS (ECS)	Must	Medium
Autoscaling ECS	Must	Medium

EPIC 3 — RAG et LangChain

User Story	Priorité	Complexité
Retriever FAISS	Must	Medium
Prompt structuré	Must	Low
Chaîne LCEL	Must	Medium

EPIC 4 — Chatbot et Frontend

User Story	Priorité	Complexité
------------	----------	------------

Chatbot CLI	Must	Low
API REST	Must	Medium
Interface web Streamlit	Nice	High

EPIC 5 — Mémoire conversationnelle

User Story	Priorité	Complexité
Stockage historique	Nice	Medium
Personnalisation	Nice	High

EPIC 6 — Géolocalisation

User Story	Priorité	Complexité
Détection localisation	Nice	Medium
Filtrage géographique	Nice	Medium

EPIC 7 — Recherche web temps réel

User Story	Priorité	Complexité
Intégration smolagent	Nice	High

EPIC 8 — Monitoring

User Story	Priorité	Complexité
Logs	Must	Low
Métriques	Must	Medium
Dashboard	Nice	Medium

4. Architecture technique détaillée

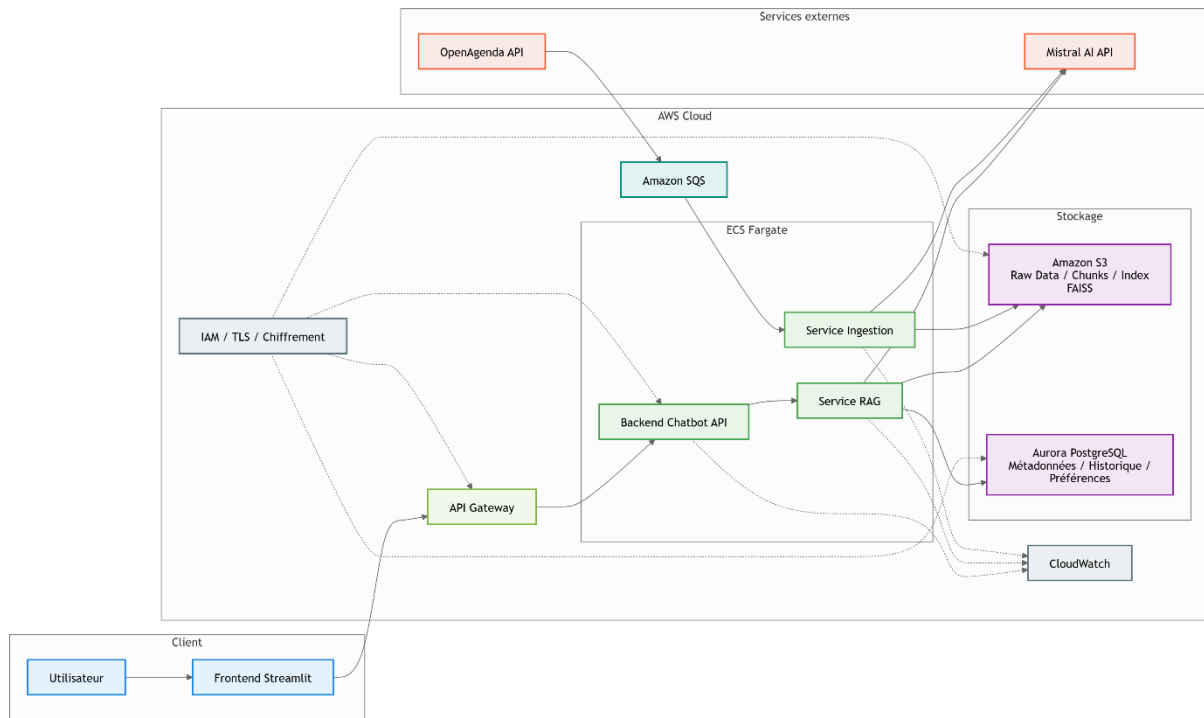
4.1 Architecture cloud (AWS)

Pipeline : Source → Ingestion → Vectorisation → Indexation → RAG → LLM → API → Frontend → Monitoring

Composants

- S3 : stockage brut + chunks + index FAISS
- SQS : file d'ingestion scalable
- ECS Fargate ingestion workers : ingestion, nettoyage, chunking, embeddings
- ECS Fargate RAG service : retrieval, prompt, appel Mistral, autoscaling
- API Gateway : endpoint chatbot
- Frontend Streamlit : interface MVP consommant l'API

- Mistral API : génération
- CloudWatch : logs et métriques
- IAM : sécurité
- Base de données :
 - Aurora PostgreSQL Serverless v2 (métadonnées, préférences, historique)
 - pgvector (option évolutive pour remplacer FAISS si besoin)



4.1.1 Scalabilité de l'index vectoriel

Problème : FAISS est très performant mais non managé, il ne scale pas automatiquement.

Solution : FAISS + ECS Fargate + S3

- L'index FAISS est généré offline et stocké sur S3.
- Chaque tâche ECS charge l'index en RAM au démarrage.
- ECS Fargate scale horizontalement selon la charge.
- Le service RAG est stateless, ce qui permet un autoscaling efficace.

Avantages :

- Performance maximale (FAISS en RAM)
- Scalabilité horizontale
- Coût maîtrisé
- Compatible LangChain + Mistral

Évolution possible :

Migration vers Aurora PostgreSQL + pgvector si :

- le volume dépasse la RAM,
- les mises à jour deviennent fréquentes,
- des requêtes hybrides SQL + vector search sont nécessaires.

4.2 Justification du cloud provider

Critère	AWS	GCP	Azure
NLP	Très bon	Bon	Moyen
Serverless	Excellent	Excellent	Bon
Monitoring	Excellent	Excellent	Bon
Coût	Bon	Moyen	Bon
Écosystème	Excellent	Bon	Moyen

Conclusion : AWS est le meilleur choix pour un MVP NLP/RAG.

4.3 Stratégie de déploiement

- Docker + ECS Fargate
- CI/CD GitHub Actions
- Versioning FAISS sur S3
- Infrastructure as Code (Terraform)

4.4 Monitoring

- CloudWatch Logs
- CloudWatch Metrics
- Dashboard : latence, coût, taux de réponse, erreurs
- Option : Datadog ou Grafana

4.5 Sécurité et RGPD

- IAM (permissions minimales)
- Chiffrement S3
- TLS
- Logs anonymisés
- Pas de stockage de données sensibles

5. Estimation des coûts

5.1 BUILD

Élément	Durée	Coût estimé
---------	-------	-------------

Ingestion	1 semaine	2 500 €
RAG	2 semaines	5 000 €
Déploiement cloud	1 semaine	2 500 €
Frontend Streamlit	1 semaine	2 500 €
Tests et optimisation	1 semaine	2 500 €

Total BUILD : 15 000 €

5.2 OPEX

LOW (100 utilisateurs/jour) : 103 €/mois MEDIUM (1 000 utilisateurs/jour) : 350 €/mois
HIGH (10 000 utilisateurs/jour) : 1 360 €/mois

5.3 Optimisation budgétaire

- Cache embeddings
- Batch ingestion
- Compression S3
- Logs filtrés
- Autoscaling ECS
- Mise en veille du frontend Streamlit hors heures de pointe

5.4 KPIs

- Latence moyenne
- Coût par 1 000 requêtes
- Taux d'erreur
- Utilisation CPU ECS
- Engagement utilisateur (frontend Streamlit)

6. Bilan

6.1 Du POC au MVP

POC : RAG local + FAISS + CLI MVP : architecture cloud scalable + monitoring + sécurité + frontend Streamlit

6.2 Justification des choix

- Mistral : cohérence embeddings + LLM
- FAISS : performance
- LangChain : orchestration
- AWS : scalabilité
- Streamlit : rapidité de mise en place pour un MVP

6.3 Défis rencontrés

- Nettoyage des données
- Chunking optimal
- Filtrage par année
- Latence API
- Coûts LLM
- Conception d'un frontend simple mais efficace

Conclusion

Le projet Puls Events évolue d'un POC technique vers un MVP robuste, scalable et prêt pour la production. L'architecture proposée permet une montée en charge progressive, un coût maîtrisé, et une intégration fluide dans la plateforme existante. Le frontend Streamlit permet une validation rapide de l'expérience utilisateur avant une intégration complète dans la plateforme Puls Events.

Les prochaines étapes incluent :

- mémoire conversationnelle avancée,
- géolocalisation enrichie,
- recherche web temps réel,
- intégration front-end complète dans Puls Events.

Annexes

Portfolio GitHub

<https://github.com/yeodramane225-cell/portfolio>

Contient :

- POC Puls Events
- MVP Puls Events
- Projets NLP
- Projets Data Engineering
- Dashboards BI

Liens utiles

OpenAgenda API Mistral AI LangChain FAISS